

ASSERTIONS

IN SYSTEM VERILOG

AKIRI JAYALAKSHMI

CONTENTS

1. Introduction to Assertions
2. Types of Assertions
3. Building blocks of Assertions
4. Action block in SVA
5. Implication operator
6. Timing window
7. Repetition operator
8. Assertions sampling methods

ASSERTIONS:

- Assertions are **statements that check design behavior automatically**.
- Used to verify if the design meets expected conditions.
- Help detect **functional bugs early** in simulation or formal verification.

WHY WE NEED ASSERTIONS:

- ✓ Manual debugging is time-consuming.
- ✓ Assertions provide **self-checking testbenches**.
- ✓ Reduce **verification effort** and improve coverage.
- ✓ Capture corner-case bugs quickly.

WHERE WE USE ASSERTIONS:

- ❖ Assertions are used **wherever correctness must be guaranteed** — in **design, testbench, simulation, or formal verification** — to **catch bugs early** and **reduce debug time**.

Types of Assertions in SystemVerilog

SystemVerilog provides **two main types** of assertions:

1.Immediate Assertions

2.Concurrent Assertions

1.IMMEDIATE ASSERTIONS

- Immediate assertions are **procedural statements** that check a condition **instantly** when the code executes.
- They are used inside procedural blocks like initial, always, or task.
- Executed in the **Active region** of simulation.
- Check **single-cycle or instantaneous** conditions.

SYNTAX:

```
assert (expression)  
    else $error("Assertion failed");
```

EXAMPLE:

```
always @(posedge clk)  
    assert (req == 1'b1)  
        else $error("Request signal missing at time %0t", $time);
```

1.1 Deferred Immediate Assertions

- ❖ Used when you want to check the condition **after all signal updates** in the current time step.
- ❖ These are two types

Type	Region	Description
Deferred Immediate (#0)	Postponed region	Checked after active events
Final Deferred Immediate (final)	End of postponed region	Checked at end of timestep

EXAMPLE:

// #0 — check at end of current timestep

```
always @(posedge clk) begin
```

```
    assert #0 (req == grant)
```

```
    else
```

```
        $fatal("Grant didn't settle by end of timestep");
```

```
end
```

// final — run once at simulation end

```
assert final (count == 0)
```

```
else
```

```
    $fatal("Simulation ended with errors");
```

2.CONCURRENT ASSERTIONS:

- Concurrent assertions are **temporal** — they check **relationships between signals over time** (multiple clock cycles).
- Synchronized to a **clock or event**.
- Written using **sequences** and **properties**.
- Used for **protocol checking, timing relationships, and temporal behavior**.
- Evaluated in the **Observed region**, using values **sampled in the Preponed region**.

SYNTAX:

```
property req_grant;  
  @(posedge clk) req |-> ##[1:2] grant;  
endproperty  
assert property (req_grant);  
//If req is high, then grant must occur within 1 to 2 cycles.
```

Inshort:

Immediate Assertions: Single-cycle checks, used inside procedural code.

Deferred Immediate Assertions: Evaluated later to avoid races.

Concurrent Assertions: Multi-cycle checks, use sequences/properties, essential in functional verification (UVM, formal).

Building Blocks of SystemVerilog Assertions

SystemVerilog Assertions are built using **three fundamental building blocks**:

1. Sequences

2. Properties

3. Assert

These blocks work together to describe and verify **temporal behavior** in hardware designs.

1. Sequence:

- A **sequence** defines an **ordered series of events or conditions over time**.
- It captures *how signals change over multiple clock cycles*.
- To describe timing relationships like “*A followed by B after 2 cycles*”.

Syntax:

```
sequence seq_name;  
    a ##1 b;           //Signal a is true, and one clock cycle later, b is true.  
endsequence
```

2.Property

- A **property** defines a **logical relationship** between sequences or expressions.
- Properties combine one or more sequences to describe protocol rules, state transitions, or timing constraints.

Syntax:

```
sequence s_ack;  
  @(posedge clk)  
    req ##[1:3] ack;           //If req is true on a clock edge, then ack must be true in the next any of 1 to 3 cycles.  
endsequence  
property p_ack;  
  s_ack;                      // The property uses the previously defined sequence  
endproperty
```

3.Assert

- assert property attaches a **property** to simulation: it continuously checks it and reports violations.

Syntax:

```
sequence s_ack;  
  @(posedge clk)  
    req ##[1:3] ack;           //If req is true on a clock edge, then ack must be true in the next any of 1 to 3 cycles.  
endsequence  
property p_ack;  
  s_ack;                      // The property uses the previously defined sequence  
endproperty  
assert property (prop_name);  // Check correctness
```


Action Blocks in SystemVerilog Assertions:

➤ An **action block** is a statement or block of code executed when an assertion evaluates.

1. Immediate assertions can trigger actions on **both success and failure**.

Syntax:

```
assert (expression)
  pass_action begin
    // Optional actions when assertion passes
    $display("Assertion passed at time %0t", $time);
  end
else begin
    // Actions when assertion fails
    $display("Assertion FAILED at time %0t", $time);
    $fatal; // Stop simulation
  end
```

2. Concurrent assertions can only trigger an action when the assertion **fails**.

Syntax:

```
assert property (property_name)
  else begin
    // Actions when assertion fails
    $display("Property FAILED at time %0t", $time);
    $fatal;
  end
```

Why Use Action Blocks:

- ✓ Log **errors, warnings, or info** (`$display, $error`).
- ✓ Terminate simulation for **critical failures** (`$fatal`).
- ✓ Debug **protocol violations or timing errors** in the design.

IMPLICATION OPERATOR:

- ❖ The implication is equivalent to an if-then structure. The left-hand side of the implication is called the “antecedent” and the right-hand side is called the “consequent.” The antecedent is the gating condition. If the antecedent succeeds, then the consequent is evaluated.
- ❖ There are 2 types of implication:
 1. Overlapped implication
 2. Non-overlapped implication

1.Overlapped implication

- The overlapped implication is denoted by the symbol `|->`.
- If there is a match on the antecedent, then the consequent expression is evaluated in the same clock cycle.

Syntax:

property p;

 @(posedge clk) a |-> b; //“a” is high on positive clock edge, then “b” should also be high on the same clock edge.

endproperty

assert property(p);

2.Non-overlapped implication

- The non-overlapped implication is denoted by the symbol $|\Rightarrow$.
- If there is a match on the antecedent, then the consequent expression is evaluated in the next clock cycle.

Syntax:

property p;

 @(posedge clk) a $|\Rightarrow$ b; //“a” is high positive clock edge, then “b” should be high on the next clock edge

endproperty

assert property(p);

Timing windows in SVA Checkers

- A **timing window** defines the interval during which an **assertion or checker is active** and can detect an event or property violation.
- $##[a:b] \rightarrow$ Checker looks **from a cycles to b cycles after a trigger**.

Syntax:

property p;

 @(posedge clk) a $|\rightarrow ##[1:4]$ b; // if “a” is high on pos edge, then within 1 to 4 clock cycles, the “b” should be high.

endproperty

assert property(p);

REPETITION OPERATOR:

Repetition operators allow specifying **how many times a condition must hold** inside a sequence. They are written inside square brackets `[*]`, `[=]`, `[->]`.

1. **Consecutive repetition** `[*]` operator is used to specify that a signal or a sequence will match continuously for the number of clocks specified.

Syntax

signal `[*n]` or sequence `[*n]`

"n" is the number of repetitions.

2.Non-Consecutive Repetition – `[=]`

Counts how many times an expression **becomes true**, not necessarily consecutive.

Syntax:

`expr[=N]` // exactly N times

`expr[=min:max]` // between min and max times

3.Go-To Repetition – `[->]`

Requires an expression to occur **at least N times**, optionally with a max limit.

Syntax:

`expr[->N]` // at least N times

`expr[->min:max]` // at least min, at most max times

ASSERTIONS SAMPLING METHODS:

Method	Meaning	Example	Explanation
1. \$rose(expr)	True when expr changes 0 → 1 in current clock cycle	`assert property (@(posedge clk) \$rose(req)	-> ack);`
2. \$fell(expr)	True when expr changes 1 → 0 in current clock cycle	assert property (@(posedge clk) \$fell(reset));	Detects falling edge of reset
3. \$stable(expr)	True if expr did not change from previous cycle	assert property (@(posedge clk) \$stable(data));	Ensures data stays constant
4. \$changed(expr)	True if expr changed from previous cycle	assert property (@(posedge clk) \$changed(addr));	Detects any change in addr
5. \$past(expr, N, clk)	Returns the value of expr N cycles before	\$past(data,1)	Used to compare current vs previous value
6. \$isunknown(expr)	True if expr has X or Z bits	assert property (! \$isunknown(signal));	Ensures no unknown values
7. \$countones(expr)	Returns number of bits that are 1	assert (\$countones(data)==1);	One-hot check
8. \$onehot(expr)	True if exactly one bit of vector is 1	assert (\$onehot(sel));	Checks one-hot encoding
9. \$onehot0(expr)	True if 0 or 1 bits are 1	assert (\$onehot0(sel));	Allows all zeros too

Thank you!

